Worksheet 5

Student Name:Suyash **UID:25MCI10054**
Branch:MCA (AI&ML) **Section/Group:MAM-1 A**
Semester:2nd **Date of Performance:03/03/2026**
Subject Name:- Technical Training -I **Subject Code: 25CAP-652**

Implementation of Cursors for Row-by-Row Processing in PostgreSQL

Aim

To gain hands-on experience in creating and using cursors for sequential row-by-row processing and applying procedural logic for complex business operations in PostgreSQL.

Tools Used

● PostgreSQL

Objectives

● To understand sequential data access using cursors

● To perform row-level manipulation using procedural logic

● To learn the lifecycle of a cursor (DECLARE → OPEN → FETCH → CLOSE)

● To handle exceptions and cursor termination conditions

● To relate cursor concepts with real-world enterprise applications

Theory

While SQL is generally set-oriented, certain tasks require a procedural approach where we process one row at a time. This is where Cursors are used:

1. **Cursor Types:** Cursors can be Implicit (managed by the system) or Explicit (defined by the developer). They can also be Forward-Only (moving only toward the end) or Scrollable (moving back and forth).
2. **The Lifecycle: * DECLARE:** Defines the SQL query for the cursor.

   OPEN: Executes the query and establishes the result set.

   FETCH: Retrieves a specific row into variables for processing.

   CLOSE: Releases the current result set.

   DEALLOCATE: Removes the cursor definition from memory.

3. **Use Case:** Cursors are ideal for generating row-specific reports, updating balances based on complex historical data, or migrating data where each record needs individual validation

Experiment Steps

Step 1 : Simple Forward-Only Cursor

```
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(50),
experience INT,
salary INT,
performance_rating INT
);

INSERT INTO employees VALUES
(101, 'Sumit', 3, 40000, 4),
(102, 'Neha', 6, 50000, 5),
(103, 'Rahul', 2, 32000, 3),
(104, 'Priya', 8, 65000, 5),
(105, 'Karan', 5, 48000, 4);
```

Table

	emp_id [PK] integer	emp_name character varying (50)	experience integer	salary integer	performance_rating integer
1	101	Sumit	3	40000	4
2	102	Neha	6	50000	5
3	103	Rahul	2	32000	3
4	104	Priya	8	65000	5
5	105	Karan	5	48000	4

```
DO $$
DECLARE
emp_record RECORD;
emp_cursor CURSOR FOR
SELECT emp_id, emp_name, salary FROM employees;
BEGIN
OPEN emp_cursor;

LOOP
FETCH emp_cursor INTO emp_record;
EXIT WHEN NOT FOUND;

RAISE NOTICE 'Employee ID: %, Name: %, Salary: %',
emp_record.emp_id, emp_record.emp_name, emp_record.salary;
END LOOP;

CLOSE emp_cursor;
END;
$$;
```

Output

```
NOTICE:  Employee ID: 101, Name: Sumit, Salary: 40000
NOTICE:  Employee ID: 102, Name: Neha, Salary: 50000
NOTICE:  Employee ID: 103, Name: Rahul, Salary: 32000
NOTICE:  Employee ID: 104, Name: Priya, Salary: 65000
NOTICE:  Employee ID: 105, Name: Karan, Salary: 48000
DO
```

Step 2: Salary Update Using Cursor (Experience + Performance Logic)

```
DO $$
DECLARE
emp_record RECORD;
emp_cursor CURSOR FOR
SELECT * FROM employees;
bonus NUMERIC;
BEGIN
OPEN emp_cursor;

LOOP
FETCH emp_cursor INTO emp_record;
EXIT WHEN NOT FOUND;

bonus := emp_record.salary *
(emp_record.experience * 0.02 +
emp_record.performance_rating * 0.03);

UPDATE employees
SET salary = salary + bonus
WHERE emp_id = emp_record.emp_id;

RAISE NOTICE 'Salary updated for Employee ID: %',
emp_record.emp_id;
END LOOP;

CLOSE emp_cursor;
END;
$$;
```

Output

```
NOTICE:  Salary updated for Employee ID: 101
NOTICE:  Salary updated for Employee ID: 102
NOTICE:  Salary updated for Employee ID: 103
NOTICE:  Salary updated for Employee ID: 104
NOTICE:  Salary updated for Employee ID: 105
DO
```

Table after step 2:

	emp_id [PK] integer	emp_name character varying (50)	experience integer	salary integer	performance_rating integer
1	101	Sumit	3	47200	4
2	102	Neha	6	63500	5
3	103	Rahul	2	36160	3
4	104	Priya	8	85150	5
5	105	Karan	5	58560	4

Step 3: Exception Handling for Empty Result Set

```
DO $$
DECLARE
emp_record RECORD;
emp_cursor CURSOR FOR
SELECT * FROM employees WHERE salary > 100000;
BEGIN
OPEN emp_cursor;
FETCH emp_cursor INTO emp_record;
IF NOT FOUND THEN
RAISE NOTICE 'No employee found with given condition';
END IF;
CLOSE emp_cursor;
END;
$$;
```

Output

```
NOTICE:  No employee found with given condition
DO
```

Learning Outcome

- Understand sequential row-by-row processing using cursors
- Implement cursor lifecycle correctly
- Apply complex business logic using procedural SQL
- Handle empty result sets and termination conditions
- Use cursors in enterprise scenarios like payroll and reporting

Result

This experiment demonstrates how cursors enable row-wise data processing in PostgreSQL.

Students learn cursor declaration, fetching, updating records, and proper resource management, which are essential for implementing complex business logic in enterprise-grade database a pplications.